

Wrapping SDL3 in Swift: Architecture and Strategy for Fountain-Coach

Module Placement Options for the SDL3 Wrapper

Option 1 – Keep the wrapper inside Teatro: One approach is to continue developing the SDL3 wrapper within the **Teatro** repository (e.g. as part of the `TeatroGUI` package). The advantage is that it stays close to the primary use case (the Teatro preview GUI) and avoids creating a new library initially. However, this tightly couples SDL support to Teatro. Other modules (like audio tools) would have to depend on the Teatro repo just to reuse SDL functionality, which could pull in unrelated code. It also conflicts with Teatro's design goal of keeping core logic headless and modular – GUI and audio components are deliberately separate ¹ ². Relying on Teatro for SDL in other contexts would reduce modularity.

Option 2 – A new shared SDL wrapper package: Creating a dedicated **Fountain-Coach SDL wrapper** repository (e.g. a `SwiftSDL3` package) is a highly modular solution. This wrapper can be developed as an independent Swift Package that any project in the ecosystem can include. It aligns with how other native libraries are handled (for example, the Csound integration is a separate `swift-csound` package with a narrow interface) ³ ⁴. A shared SDL package would serve as a single source of truth for all SDL interactions, avoiding duplication. The downside is the overhead of maintaining another package, but given Fountain-Coach's modular architecture it's likely worthwhile. This approach keeps **Teatro** focused on rendering logic while the SDL wrapper evolves on its own schedule. It also makes it easier to eventually open-source or update the SDL layer independently.

Option 3 – Split SDL logic across existing modules: In theory, SDL functionality could be added piecemeal into the modules that need it – for example, adding SDL audio code directly into the `midi2sampler` or `swift-csound` packages for sound output, and keeping windowing code in Teatro. This avoids a new dependency, but it sacrifices consistency. Each module would end up wrapping SDL in its own way, duplicating effort and making upgrades harder. It would be better to **centralize SDL abstractions** so fixes and improvements benefit all uses. Given that TeatroGUI already defines window, event, and rendering stubs ready to accept a real SDL implementation ⁵ ⁶, it's logical to generalize these into a shared library rather than copy them elsewhere.

Recommendation: Use a **new shared repository** for the SDL3 wrapper. This library can house all SDL-related abstractions (windowing, events, rendering, audio, etc.) in one place, designed for reuse. Teatro's GUI package can then depend on this wrapper (replacing its internal stubs), and other packages can pull it in as needed. This approach maximizes modularity and reusability, consistent with Fountain-Coach's architecture. It prevents unintended coupling (e.g. `midi2sampler` needing to import Teatro) and ensures that SDL specifics are maintained in one well-defined module. The Teatro repo can slim down by removing SDL C integration details, depending on the wrapper for actual SDL calls. In summary, a standalone SDL wrapper package provides a clean separation of concerns and makes cross-platform support easier to manage for all projects.

Designing an Idiomatic and Swiftly SDL3 Wrapper

The SDL3 wrapper should expose a **Swift-friendly API** rather than mirroring C functions. The goal is to make it feel “Swiftly” – type-safe, memory-safe, and using Swift conventions – while still providing low-level control. Key design considerations include:

- **High-level types instead of raw pointers:** Wrap SDL constructs (window, renderer, surface, audio device, etc.) in Swift classes/structs. For example, an `SDLWindow` class can internally hold a pointer to the `SDL_Window` but expose methods and properties to manage it. The current Teatro stub uses an `SDLWindow` class with an `open()` and `close()` method, which in the real implementation would call `SDL_CreateWindow` and `SDL_DestroyWindow` respectively ⁷ ⁸. This encapsulation hides C pointers; the public API deals with Swift objects.
- **Use `OptionSet` and `Enum` for constants:** Replace C bitmask flags and numeric constants with Swift `OptionSet` and `enum` types. For example, SDL defines flags for window creation (fullscreen, resizable, etc.) – the wrapper can define a Swift `WindowOptions: OptionSet` for these. Likewise, SDL’s event types or key codes can be represented as Swift enums. In Teatro’s stub, they already created an `SDL_Event` enum (cases for `.quit`, `.keyDown`, `.mouseMove`, etc.) instead of exposing SDL’s integer event codes ⁹. This makes code more readable and type-safe. Enums and option sets also allow using Swift’s pattern matching and set algebra rather than bit-twiddling C defines, aligning with Swift best practices ¹⁰ ¹¹.
- **Safe memory management (RAII):** The wrapper should manage SDL resources in a RAII-style (Resource Acquisition Is Initialization) to ensure no leaks or crashes. Each Swift wrapper class can take responsibility for initializing and cleaning up its underlying SDL resource. For instance, an `SDLRenderer` class could call `SDL_CreateRenderer` on init and `SDL_DestroyRenderer` on deinit. Similarly, an `SDLSurface` or `SDLTexture` struct could free its memory in deinit. By tying SDL lifecycle to Swift object lifecycle, we prevent common C pitfalls like forgetting to free resources. The API can also use Swift `defer` or guard statements to handle error cleanup if SDL calls fail partway.
- **Error handling with Swift `Result` / `Error`:** SDL functions typically return error codes or `NULL` and require calling `SDL_GetError()` for a message. The Swift wrapper should instead throw Swift `Error` exceptions or return `Result` types. For example, `SDLWindow.open()` can be written to throw if window creation fails, embedding the SDL error string in a Swift error type. This way, SDL errors are handled in the Swift-error paradigm rather than requiring users to manually fetch C error strings. It makes the API more idiomatic and encourages proper error handling.
- **No global state leakage:** Keep SDL’s global state (initialization flags, etc.) behind the scenes. For instance, the wrapper might internally call `SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO)` as needed, but the user of the wrapper wouldn’t have to call `SDL_Init` directly. The first use of an SDL feature could trigger initialization internally (lazy init) or the wrapper could provide an explicit `initialize(subsystems:)` method if needed. In either case, avoid exposing the C SDL init flags – instead use a Swift struct or `OptionSet` if the user needs to specify subsystems. The **Teatro design explicitly wants SDL details not to leak into public API** ¹², meaning external modules should not have to import or understand SDL’s C API at all.
- **Thread safety and Swift concurrency:** SDL is often used from a single thread (usually the main thread for graphics). The wrapper classes can be made thread-safe where appropriate or marked

as confined. In Teatro's stub, `SDLWindow` is marked `@unchecked Sendable` and uses an internal `DispatchQueue` for event handling ¹³ ¹⁴. The real implementation should similarly ensure thread-safe access to event queues or use locking if needed (for example, pushing events from background threads). Designing the API to work nicely with Swift concurrency (like async tasks) is also valuable – e.g., providing async initialization or using a loop that can be cancelled. However, since SDL itself isn't async-aware, a simple approach is to keep SDL calls on one thread and use Swift concurrency only at higher levels (like signaling an async task when an SDL event occurs).

- **Swift naming and conventions:** Follow Swift naming guidelines for methods and types. For example, instead of `SDL_PollEvent`, the wrapper might use a method `window.pollEvent()` returning an `SDL_Event?`. The naming in stubs (`SDLRenderer.drawRect(x:y:w:h:color:)` etc.) already follows lowerCamelCase method naming ¹⁵. Properties should use Swift types (e.g. use `Int` for sizes, `Bool` for flags like vsync). Also prefer Swift's value types and collection types over C arrays/pointers – e.g., returning a `[UInt8]` or `Data` for pixel data rather than an `UnsafePointer<UInt8>`.
- **Minimal overhead abstraction:** Although the API should be high-level, it should remain a thin layer over SDL to avoid performance penalties. Expensive operations (like pixel access) can be designed carefully (perhaps allowing direct pointer access to a pixel buffer when needed, but safe by design). The wrapper can also expose some SDL constants or functions directly if there's a use-case not covered by a high-level construct, but those can be carefully vetted to not break the idiomatic feel.

By adhering to these design principles, the wrapper will let developers use SDL in Swift as naturally as using a pure Swift framework. For example, opening a window and drawing might look like:

```
let window = SDLWindow(title: "Demo", width: 800, height: 600)
try window.open() // internally calls SDL_CreateWindow
let renderer = try SDLRenderer(window: window) // SDL_CreateRenderer
renderer.clear(color: .black)
renderer.drawRect(x: 50, y: 50, width: 100, height: 100, color: .green)
renderer.present() // internally SDL_RenderPresent
```

This pseudocode illustrates a clear, Swift-style flow (initializers, throwing errors, methods on objects) instead of the C-style `SDL_CreateWindow`, `SDL_CreateRenderer`, `SDL_SetRenderDrawColor`, ... sequence with global functions. Internally, each call would manage C interop, but the outer appearance is safe and Swift. Teatro's current preview controller code already follows a similar pattern with an `SDLWindow`, `SDLRenderer`, and `SDLRunLoop` object orchestrated in Swift ¹⁶ ¹⁷ – the difference is that today those are stub implementations. Moving forward, those classes will become real SDL-backed implementations with the same external API.

Crucially, **resource cleanup** should be automated. For example, if an `SDLAudioDevice` class opens an audio device via `SDL_OpenAudioDevice`, it should close it in `deinit` or via a `close()` method (with `deinit` as a fallback). This ensures no SDL resources remain open if Swift objects go out of scope. Swift's deterministic deinitialization is useful here.

Finally, maintain alignment with **Swift evolution features**: for instance, if new Swift features (like improved C interop or an official `@OptionSet` macro) become available, the wrapper can adopt them

to reduce boilerplate. But initially, a manual definition of OptionSet and enums will be needed for SDL3's various flags.

Implementation Scaffolding and SDL Module Layout

To implement the wrapper cleanly, leverage the Swift Package Manager's support for mixed-language packages. A possible module layout could be:

- **C SDL System Library Target:** Include a target (say `CSDL3`) that provides the raw SDL3 C headers to Swift. This is done by adding a module map that imports SDL3's headers (SDL.h and related). On Apple platforms, this can be linked to an XCFramework or a vendored binary; on Linux, it can reference the system-installed SDL3. For example, the package might require developers to install SDL3 via Homebrew on macOS (or use a bundled binary) and `libSDL3-dev` on Linux. The `Package.swift` for the wrapper can declare `CSDL3` as a **system library** target for SDL3. Similarly, if `SDL_ttf` is needed, a `CSDL_ttf` system library target can be added to expose `SDL_ttf`'s C API.
- **Binary vs. source linking:** For convenience on platforms like macOS and iOS, providing a pre-compiled **SDL3 XCFramework** can simplify integration (no manual brew install). For instance, an existing SDL2 Swift package includes an XCFramework so that “no extra steps are needed” on Apple platforms ¹⁸. We can do the same for SDL3: bundle an `SDL3.xcframework` (and perhaps `SDL_ttf.xcframework`) that contains builds for macOS (arm64/x86_64) and iOS. The Swift package can use a binary target to include these. On Linux, we'd detect the platform and fall back to a system library (expecting the developer/CI to install SDL via the package manager) ¹⁸. On Windows, we could either include import libraries or instruct manual setup (since Swift on Windows is still evolving, an included `.dll` or build script may be needed, as seen in other projects).
- **Swift Wrapper Target:** The main Swift target (call it something like `SDLWrapper` or `FCSDLKit`) will depend on the C targets. It will contain all the Swift code (the classes, enums, etc. discussed above). In `Package.swift`, this target would specify dependencies on `CSDL3` (and `CSDL_ttf` if used) so it can call SDL's C functions. This target will produce a module (e.g. importable as `SDLWrapper`) that clients use. The public API of this module is the Swifty interface; it internally uses `import CSDL3` to call into SDL.
- **Integration of SDL_ttf and others:** If text rendering (`SDL_ttf`) is needed, we can either integrate it into the same wrapper target or provide it as an add-on. One approach is to include font support in the main wrapper: for example, have an `SDLFont` class that uses `SDL_ttf` functions to load fonts and render text to surfaces. The wrapper would call `TTF_Init()` under the hood when needed. Alternatively, we could create a separate target (e.g. `SDLTTF`) that depends on the core wrapper. This would allow not linking `SDL_ttf` if a given project doesn't need it. In practice, the overhead of linking `SDL_ttf` is small, so bundling it is fine, but splitting it out keeps the core lean. The decision can be based on how frequently text rendering is expected in the ecosystem. Since **Teatro** likely will need text for drawing annotations or lyrics, it makes sense to include `SDL_ttf` support from the start. The package can initialize `SDL_ttf` on-demand and expose a Swifty API for text (e.g. `SDLFont.render(text:String, size:Int) -> SDL_Surface`). The same pattern could extend to other subsystems (`SDL_image` for loading images, `SDL_mixer` for advanced audio) as needed – each either integrated or as optional targets.

- **Conditional compilation for stubs:** To maintain the ability to build and test without SDL present (e.g. continuous integration or Swift playgrounds), the wrapper can use conditional compilation to provide **no-op stubs** when the actual SDL libraries aren't available. Swift can conditionally compile code with `#if canImport(CSDL3)`. For example, the implementation of `SDLWindow.open()` could be written in two versions: one compiled when `CSDL3` is available (calling `SDL_CreateWindow`), and one when it isn't (perhaps just logging or throwing an error). Another strategy is to have two separate Swift targets: a "real" SDL implementation target and a "stub" target. However, that complicates the API exposure. A simpler route is a compile-time flag or the presence of the C library to switch behavior. The Fountain-Coach approach for Csound is a good reference: the `SPCsoundBackend` target is only built if the Csound binary is present, otherwise a stub implementation (that throws on use) is used so that the package still resolves¹⁹. We can emulate this: for example, define `SDL_AVAILABLE` when SDL libs are linked, and compile dummy implementations when not. This ensures developers can build the project even if they haven't installed SDL, and our CI can run headless tests without a graphics driver.
- **SwiftPM package setup:** The package manifest would list the C targets and the Swift wrapper target. Downstream projects would add this package via Git URL and then depend on the product (e.g. `.product(name: "SDLWrapper", package: "swift-sdl3-kit")`). The package can be configured to **automatically link SDL libraries** on each platform. For instance, on Apple, the binary XCFramework could contain the SDL3 dynamic library; on Linux, the `pkgConfig` or `providers` in the system library manifest can help SwiftPM find `-lSDL3`. We should also document any setup needed (like `apt-get` commands). In the SDL2 wrapper example, developers are instructed to install SDL2 on Linux via `apt`²⁰ – we'd provide similar instructions for SDL3.

To illustrate the structure, here's a conceptual layout:

- **Package Name:** `FountainSDL` (for example)
- **Targets:**
 - `CSDL3` – system library target for SDL3 (module map pointing to SDL3 headers; links `SDL3` library).
 - `CSDL_ttf` – system library target for SDL_ttf (module map for SDL_ttf headers; links `SDL3_ttf`).
 - `SDLWrapper` – Swift target that depends on `CSDL3` (and `CSDL_ttf`). This contains `SDLWindow`, `SDLRenderer`, `SDLEvent`, `SDLAudioDevice`, `SDLFont`, etc. Public API.
- **Products:**
 - `.library(name: "FountainSDL", targets: ["SDLWrapper"])` – the product that packages can import.

By organizing it this way, adding the dependency in another project is straightforward. For example, in **Teatro's Package.swift**, they would replace the current implicit SDL stub with a dependency:

```
.package(url: "https://github.com/Fountain-Coach/FountainSDL.git", from:
"1.0.0"),
...
.target(name: "TeatroGUI", dependencies: [
    .product(name: "FountainSDL", package: "FountainSDL"),
    ... other deps ...
]),
```

Then in code, `TeatroGUI` would `import FountainSDL` (or whatever the module is named) and gain access to the `SDLWindow`, `SDLRenderer`, etc., now backed by real SDL. The transition for Teatro would involve minimal code changes – mostly replacing the stub implementation with the real one in the wrapper package and deleting any stub-specific scaffolding that’s no longer needed in Teatro.

One important integration detail is **initialization and teardown**: SDL requires `SDL_Init` (with flags) and `SDL_Quit`. The wrapper can handle this by automatically calling `SDL_Init` for required subsystems when the first SDL object is created. For instance, creating an `SDLWindow` might ensure `SDL_Init(SDL_INIT_VIDEO)` has run. The wrapper could also provide an explicit `SDL.initialize(flags:)` if needed, but hiding it is cleaner. On exit, it should call `SDL_Quit` when the program ends – possibly via `atexit` or simply documented that the process will quit SDL on termination. Ensuring this happens exactly once is important (`SDL_Init` is safe to call multiple times, but `SDL_Quit` should match). A simple reference count or static boolean in the wrapper can guard the init/quit calls.

Finally, **SDL_ttf integration** requires calling `TTF_Init()` before using font functions and `TTF_Quit()` at the end. The wrapper should handle this similarly (maybe the first creation of an `SDLFont` triggers `TTF_Init()`). These subsystems can be managed behind the scenes in the wrapper so the user doesn’t have to call multiple init functions.

By using SwiftPM and system libraries or binary targets, we keep the third-party C dependency (SDL) manageable. This approach has been validated by both the project’s own Csound integration (which used an xcframework and system libs) and community packages like `SwiftSDL2` ¹⁸. The end result is a robust scaffolding where adding SDL to any module is as easy as adding the package and writing Swift code – no manual C glue needed per project.

Public API Surface for Broad Reuse

Designing the wrapper’s **public API** with reusability in mind means making it **independent of Teatro-specific concepts** and suitable for any Swift module that needs multimedia support. The API should be **clean and minimal**, exposing only what’s necessary to use SDL’s capabilities in Swift, and nothing tied to Fountain-Coach domain models.

Key guidelines for the public API surface:

- **Neutral naming:** The module and types should be named for SDL functionality, not for Teatro. For example, `SDLWindow`, `SDLEvent`, `SDLAudioDevice` are clear, generic names that any developer could understand. Avoid names like “TeatroWindow” or anything implying a specific usage. If the wrapper is a separate package, it might be named something like `SDL3Kit` or `FountainSDL` – the name should reflect it’s a general SDL3 binding for Swift.
- **Focus on SDL abstractions:** The public API should include types for the main SDL constructs (windows, renderers, surfaces/textures, events, perhaps game controllers and audio devices if needed). It should **not** include things outside SDL’s scope. For example, while `TeatroPreviewController` is a higher-level concept that ties SDL to the Teatro rendering loop, that controller itself would remain in Teatro (using the wrapper under the hood) ²¹. The wrapper’s API doesn’t need a “PreviewController”; it provides the building blocks (windows, etc.) for Teatro or any app to create such controllers. In short, keep the wrapper API low-level enough to be flexible, but high-level enough to be ergonomic.

- **Hide SDL C types:** As mentioned earlier, do not expose `UnsafePointer`, `SDL_*` structs, or C enums in the public API. All interactions should be via Swift types. For instance, if a function returns an `SDL_Surface*` internally, the public API should return an `SDL_Surface` object or perhaps a Swift `Image` type (if we convert the surface to a higher-level image). This ensures that downstream modules never have to include `<SDL.h>` or deal with raw pointers. The **Teatro design already enforces this – SDL specifics are encapsulated and not exposed** ¹², so the wrapper should maintain that discipline for all consumers.
- **Permit extension and composition:** The API should be open enough that other modules can build on it. For example, if `midi2sampler` wanted to use the SDL wrapper to play audio, it might need an `SDLAudioDevice` interface that lets it feed audio buffers. The wrapper could provide a callback-based audio playback API (where you supply a Swift closure to generate audio samples). Downstream, `midi2sampler` can then plug in its DSP code into that closure. Similarly, if a downstream app wants to handle custom events, the `SDL_Event` enum might need to include a case for controller input or other events SDL supports, even if Teatro doesn't use them yet. Designing for reuse means thinking slightly beyond Teatro's immediate needs and including the broader SDL feature set where feasible (at least in skeleton form). We don't necessarily implement everything at once, but the public API shouldn't be a dead-end that only works for one scenario.
- **Documentation and discoverability:** Because this wrapper could be used by other developers or future team members in different projects, comprehensive documentation comments and examples are important. Each public class and function should be documented (in code) to explain usage, any performance considerations, and how it maps to SDL concepts. This will make the API approachable even outside the context of Teatro. It might also be worth providing a few example snippets (in the repository README or docs) for common tasks (window creation, drawing shapes, playing a sound) to illustrate how the API is used.
- **Stability and versioning:** As a shared module, the SDL wrapper should be versioned and released so that different packages can depend on a specific version. This means once the public API is decided and in use, breaking changes should be managed via semantic versioning. This is a change from the in-repo stubs which could be modified freely. A stable API surface will allow multiple modules (Teatro, and others) to rely on it without fear of sudden changes. Of course, during initial development it can start unstable (0.x version), but the goal is to converge to a stable interface that can be widely adopted.

In practice, the **Teatro** module would become just one consumer of this public API. For example, the `TeatroPreviewController` in Teatro currently instantiates `SDLWindow`, `SDLRenderer`, etc. directly ²². After factoring out the wrapper, Teatro will import the SDL package and do the same, except now those classes come from the shared module. Because we plan the API to be general, a hypothetical **midi2sampler player** or a **visualizer tool** could also import the SDL package and use those same classes for their UI or audio without pulling in Teatro. This clean separation means **wider exposure**: any downstream module that needs SDL's functionality can get it via this one consistent API.

Notably, if there's a need to expose some SDL wrapper functionality through FountainKit or other high-level frameworks, it can be done in a controlled way. For instance, if FountainKit (the AI/planning layer) at some point needed to trigger a GUI or audio, it could rely on a well-defined interface provided by this SDL wrapper rather than reinventing one. Having a single module responsible for "platform media" makes it clearer how other parts of the ecosystem should interact with the windowing or audio capabilities.

In summary, the public API should be **self-contained and generic**, turning SDL into a Swifty module that any part of the Fountain-Coach ecosystem (and potentially beyond) can use. By keeping it free of Teatro-specific assumptions and carefully managing the exposed surface, we ensure the module is truly reusable.

Leveraging SDL in Teatro and Midi2Sampler (Current State and Needs)

Examining the current codebases highlights where a shared SDL wrapper would immediately bring benefits:

- **Teatro (GUI preview):** Teatro already defines a skeletal SDL interface in its `TeatroGUI` module. Classes like `SDLWindow`, `SDLRenderer`, `SDLEvent`, and `SDLRunLoop` exist as *stubs* – they mimic what the real SDL calls will do, but without actually calling SDL (for instance, drawing to an offscreen buffer instead of a window) ²³ ²⁴. This was done to allow development and testing of the GUI loop without requiring SDL libraries or a display ⁵ ²⁵. The plan is to **swap in real SDL3 behind these abstractions** when ready ⁶. Adopting a shared wrapper will accomplish that swap uniformly. The Teatro code that currently enqueues synthetic events and uses a dummy pixel buffer can be removed in favor of the wrapper’s real event polling and rendering calls ¹⁷ ²⁴. Importantly, Teatro’s design avoids leaking SDL details outside (`SDLEvent` is internal and simple ⁹, and the preview is launched through a high-level controller ²⁶), so integrating the wrapper won’t force any changes on Teatro’s public API. It will simply **replace the implementation**: when the preview runs, it will open a true window and render via GPU instead of a no-op. This yields an immediate improvement – the “SDL-based preview GUI” mentioned in Teatro’s README can become reality ¹ ²⁷.
- **Midi2Sampler (audio playback):** The `Midi2SamplerDSP` package is primarily about audio processing (looping, crossfades, etc.), and not a UI. However, one likely use-case is to **audition samples or synthesized notes** in real time, especially on platforms beyond macOS. Currently, audio output in the Fountain-Coach ecosystem is handled in `TeatroAudio`, using platform-specific sinks: e.g. `AVFoundation` on macOS, `FluidSynth` or `Csound` for synthesis on multiple platforms ²⁸. If `Midi2Sampler` or related tools want a cross-platform audio output (for example, a command-line tool to quickly play a sample or a note on Linux), SDL’s audio subsystem could be very handy. SDL provides a simple way to open the default audio device and stream audio. Instead of writing separate code for `ALSA/PulseAudio` on Linux, `CoreAudio` on Mac, etc., the SDL wrapper could offer an `SDLAudioDevice` API that abstracts this. Then `Midi2Sampler` or **Fountain-Play** (if one exists) could play audio through SDL with minimal effort.

Currently, `Midi2Sampler` might not integrate audio output directly (it might hand off buffers to whoever calls it, or rely on `Csound/FluidSynth` to produce sound). But having an **SDL audio utility** available means, for example, a developer could quickly write a small player that feeds `Midi2Sampler`’s output into an SDL audio callback. This would be especially useful in a testing scenario or on Linux where `AVFoundation` is not available. In short, while `Midi2Sampler` doesn’t embed SDL now, it stands to gain a platform-neutral audio output path by leveraging the new wrapper.

- **Other potential reuse:** The Fountain-Coach ecosystem may include other tools or future modules (for instance, a visualizer for MIDI data, or an interactive editor). Any such tool that needs to create a window, draw graphics, or handle input would benefit from not having to rewrite low-level code. They could all rely on the single SDL wrapper. For example, if there were a plan to build a small waveform viewer for sample files (perhaps as part of `Midi2Sampler`’s

toolkit), using `SDLWindow` and `SDLRenderer` from the wrapper would provide a quick way to open a window and draw waveforms using SDL's 2D drawing or OpenGL. This avoids each tool using different frameworks (AppKit on Mac, something else on Linux, etc.). A unified approach also means consistency – the behavior of windows or input in all tools will be the same, because it's the same underlying SDL code.

- **Consistent event handling across modules:** Teatro's preview deals with keyboard and mouse events (e.g., `.quit` on window close, or key presses) ²⁹. If any other module needed to handle input (say a future interactive console or a game-like simulation), they could use the same `SDL_Event` constructs. The wrapper will parse native SDL events into the `SDL_Event` enum. Having that in one place means if SDL3 changes or if we need to add more event types (controller input, touch events, etc.), we do it once in the wrapper and all consumers benefit. This is better than, for instance, Teatro and another app both writing their own event translation logic.

In summary, **Teatro will directly replace its stubbed backend with the shared SDL wrapper**, immediately gaining real windowing and rendering on macOS and Linux (and even Windows later) with minimal code changes. The stub was crucial for early development and testing ²⁵, but the real SDL integration will bring the GUI to life. Meanwhile, **Midi2Sampler and audio-focused components** can start to use SDL for audio output or any simple GUI needs, ensuring they remain cross-platform. Even if Midi2Sampler's core remains headless, the existence of an audio output API in the wrapper provides an easy path to play or monitor audio in tools around it.

It's worth noting that FluidSynth (used in TeatroAudio) and Csound both have their own audio drivers, but integrating them with SDL could simplify things. For example, instead of using FluidSynth's JACK or CoreAudio drivers, one could use FluidSynth in "audio callback" mode and output via SDL. This might not be immediately necessary, but the wrapper makes such refactoring possible if it ever becomes desirable (for consistency or to reduce dependencies on other audio subsystems).

By evaluating these current implementations, it's clear that a shared SDL wrapper addresses a **common need for abstraction**: whether it's GUI or audio, multiple parts of the ecosystem need a thin platform layer. Implementing it once means Teatro's GUI gets solidified and other modules can tap into it with far less effort than doing their own thing.

Rollout Strategy and Cross-Platform Adoption

Adopting the new SDL3 wrapper across the Fountain-Coach ecosystem should be done incrementally to ensure stability:

1. Initial development and testing: Start by implementing the wrapper in a separate branch or repository, while keeping Teatro's current stub in place as a fallback. Focus first on macOS (since that's the primary development platform currently – Teatro targets macOS 14+ ²⁷). Verify that the wrapper can create a window, render a simple shape, and handle input events on macOS. During this phase, use a feature flag in Teatro (or a SwiftPM conditional) to switch between the stub and the real implementation. For example, a compile-time flag `USE_SDL` could toggle whether `SDLWindow.open()` calls the real `SDL_CreateWindow` or the stub logic. This lets you run the preview in two modes for testing.

2. Integrate with Teatro under a flag: Once the basic wrapper is working on macOS, integrate it into TeatroGUI. Guard it so that CI and headless test runs still use the stub. This can be done by enabling the

real SDL path only when an environment variable is set or a specific Swift build configuration is used. The documentation already indicates the SDL path is under a feature flag ⁵, so use that mechanism. Then manually run `swift run TeatroPreviewDemo` on a Mac with SDL available, and confirm that an actual window appears and the demo rectangle animates (as intended in the PreviewController code). This validates the end-to-end plumbing with real SDL. Also verify that calling `TeatroPreviewController` in a headless environment (without the flag) still runs offscreen as before – so tests remain deterministic.

3. Expand platform support (Linux, Windows): With macOS functioning, move to **Linux**. Ensure you have SDL3 installed on your Linux dev environment or CI (e.g., `apt install libsdl3-dev` or the appropriate library if using SDL3). If a display is available, test running the preview to see that a window opens on Linux and the same demo works. Headless CI is trickier – to test GUI on Linux CI, you might use Xvfb (X virtual framebuffer) as hinted in the docs ³⁰. Indeed, enabling Xvfb in CI allows SDL to create a renderer without a physical display/GPU. Configure the CI to start Xvfb and set `DISPLAY`, then run a simple test (perhaps open and close a window, or render one frame) to ensure no crashes. If Xvfb is not feasible, you might rely on stub mode for most CI tests and do manual Linux GUI testing in a VM or separate machine. Also, consider Windows if it's in scope: set up a Windows VM or GitHub Action with SDL installed. Windows support might require tweaking the build (for example, linking to `SDL3.dll` and using the `.xcframework` or `.lib` as needed). The SwiftSDL2 project demonstrated Windows support with a build script ³¹ – a similar approach or precompiled binary can be used. While Windows and iOS are not immediate targets, validating that the wrapper compiles for them (even if not fully tested) ensures future compatibility. SDL officially supports iOS and Windows ³², so in principle the wrapper should work on those platforms too.

4. Integrate audio subsystem: If the project roadmap includes using SDL for audio output on some platforms, implement and test the audio part of the wrapper. For instance, add an `SDLAudioDevice` that opens the default audio output. Write a test that fills a buffer with a sine wave and plays it for a second. On macOS, this should output sound (assuming audio is not otherwise occupied). On Linux, do the same – you may need to ensure the SDL audio driver finds an output (PulseAudio/ALSA installed). This step ensures the audio portion is solid before others depend on it. If `midi2sampler` is going to use it, perhaps create a small demo that uses `midi2sampler`'s output and feeds it to `SDLAudioDevice` as a proof of concept.

5. Deprecate stubs and flip the switch: Once real SDL3 support is tested on all relevant platforms and in Teatro, plan to make it the default. The documentation suggests the preview GUI will be enabled with native SDL once ready ⁶. At that point, you can remove or isolate the stub. For example, you might keep a stub around for CI only (if truly needed), but otherwise have TeatroGUI always use the real wrapper. Update Teatro's documentation to reflect that the GUI now requires SDL3 (and how to set it up on each platform). This might involve instructing users to install SDL3 or bundling the SDL library with any distributed app. Since the wrapper package will have its own instructions (like "install via brew or use the included framework"), refer to those.

6. Roll out to other modules: After Teatro, consider where else to adopt the wrapper: - **TeatroAudio or midi2sampler:** If using SDL audio improves cross-platform support, integrate it. For example, on Linux, instead of relying solely on Csound for audio out, the code could open an `SDLAudioDevice` and stream audio buffers (possibly those generated by FluidSynth or the Midi2Sampler DSP). This could simplify audio output logic by using one unified mechanism. On macOS, you might continue using AVAudioEngine for low-latency output (since it's well-integrated), but SDL could be a fallback or option. - **Utility tools:** If there are small tools or demos (CLI tools, etc.), convert them to use the SDL wrapper for any media functionality. For instance, if there's a CLI that visualizes MIDI in real-time, have it use

`SDLWindow` instead of, say, printing ASCII art or requiring a browser. - **Future projects:** Encourage using the SDL wrapper for any new feature requiring graphics or sound. By demonstrating it in Teatro, its usefulness will be clear. This prevents divergence (where one team might consider using a different library for another GUI).

7. Ensure modularity and testability throughout: As SDL capabilities get integrated, maintain the modular boundaries. The SDL wrapper should remain independent – e.g., it should not start depending on Teatro types or FountainKit logic. Conversely, higher-level packages should treat it like any other dependency. For testing, continue to provide means to run in headless mode. You can keep an **explicit stub mode for tests**: for example, if an environment variable `SDL_HEADLESS` is set, the wrapper could skip actual initialization and render to an internal buffer (similar to current behavior) ²⁴. This could be implemented by having the wrapper check for that flag in its init routines. In CI, you'd set `SDL_HEADLESS=1` to avoid trying to open a window or audio device. This way, your test suite can test the logic (even call rendering functions) without a real output. Some tests can even verify that drawing calls produce expected pixel values via the offscreen buffer hash technique already used ³³. And for testing the real integration, you might include a small integration test that is only run in an environment where a display is available (tag it separately).

8. Platform compatibility and distribution: When the wrapper is ready for prime time, make sure to package it nicely for all platforms. For Apple platforms (macOS, iOS), if you include an XCFramework for SDL, verify codesigning and bitcode (if iOS) as needed. On Linux, update documentation or automation to install SDL libraries on developer machines and CI. If releasing any binary apps (say a .app for macOS or a Linux AppImage), include the SDL library accordingly (SDL is Zlib-licensed ³⁴, which is permissive, so bundling is allowed). On Windows, distribute the SDL DLL with the executable if needed. Essentially, treat SDL similar to how you handled Csound: provide clear steps or artifacts so that end-users of the software aren't stuck figuring out dependencies.

Throughout the rollout, **communication is key**: update README and docs in each repository to note the change. For example, Teatro's README can remove the note about “SDL3 preview available with a stubbed backend” and instead say “SDL3-powered preview available (requires SDL3 library)” ²⁷. Provide a link to the SDL wrapper package for more info. This helps any contributor or user understand the new requirement.

By following this staged rollout, you minimize disruptions. Teatro keeps functioning during development (thanks to the stub fallback) and only switches to real SDL when it's verified. Other modules adopt the wrapper gradually, which limits the scope when debugging issues. In the end, the ecosystem gains a powerful, common foundation for all UI and audio tasks: SDL3 via a Swift-friendly layer. This ensures **modularity (one central SDL module)**, **testability (stub mode and headless testing remain possible)**, and broad **platform compatibility** (SDL is known to work on macOS, Linux, Windows, iOS, etc., so the wrapper inherently carries that portability). The outcome will be a more unified codebase where each part can rely on a well-tested SDL abstraction for any low-level media needs, rather than reinventing those wheels.

Overall, this strategy future-proofs the Fountain-Coach ecosystem by providing a solid multimedia backbone in Swift that can grow and be reused wherever needed. With careful architecture and planning, wrapping SDL3 in Swift becomes a catalyst for both improved functionality in Teatro and new possibilities in other modules – all while adhering to Swift idioms and the project's modular philosophy.

Sources:

- Fountain-Coach Teatro README (modular design and SDL preview status) 27 2
- Teatro Interactive GUI Chapter (encapsulation of SDL and stub-to-real transition) 12 6
- Teatro SDLBackend stubs (current SDLWindow, SDLEvent, SDLRenderer implementations) 9 24
- Fountain-Coach swift-csound README (package layout and optional C backend pattern) 4 19
- ctref's SwiftSDL2 project (example of Swift SDL package and cross-platform setup) 18 32

1 2 25 27 README.md

<https://github.com/Fountain-Coach/Teatro/blob/f92db6384ae785adeaa3b301b62ba31f35a5e3d6/README.md>

3 4 19 README.md

<https://github.com/Fountain-Coach/swift-csound/blob/324d9ed6055c4fa4893fc79c454f939b98dd8fa6/README.md>

5 6 12 21 26 30 TeatroInteractiveGUI.md

<https://github.com/Fountain-Coach/Teatro/blob/f92db6384ae785adeaa3b301b62ba31f35a5e3d6/Docs/Chapters/TeatroInteractiveGUI.md>

7 8 15 16 17 22 PreviewAPI.swift

<https://github.com/Fountain-Coach/Teatro/blob/f92db6384ae785adeaa3b301b62ba31f35a5e3d6/Sources/TeatroRenderAPI/PreviewAPI.swift>

9 29 SDLEvent.swift

<https://github.com/Fountain-Coach/Teatro/blob/f92db6384ae785adeaa3b301b62ba31f35a5e3d6/Packages/TeatroGUI/Sources/TeatroGUI/TeatroSDLBackend/SDLEvent.swift>

10 OptionSet in Swift explained with code examples - SwiftLee

<https://www.avanderlee.com/swift/optionset-swift/>

11 Understanding Swift's OptionSet - Donny Wals

<https://www.donnywals.com/understanding-swifts-optionset/>

13 14 SDLWindow.swift

<https://github.com/Fountain-Coach/Teatro/blob/f92db6384ae785adeaa3b301b62ba31f35a5e3d6/Packages/TeatroGUI/Sources/TeatroGUI/TeatroSDLBackend/SDLWindow.swift>

18 20 31 32 34 GitHub - ctref's/SwiftSDL2: Swift wrapper around Simple Direct Media Layer (SDL2) for macOS, iOS, Linux and Windows

<https://github.com/ctref's/SwiftSDL2>

23 24 33 SDLRenderer.swift

<https://github.com/Fountain-Coach/Teatro/blob/f92db6384ae785adeaa3b301b62ba31f35a5e3d6/Packages/TeatroGUI/Sources/TeatroGUI/TeatroSDLBackend/SDLRenderer.swift>

28 README.md

<https://github.com/Fountain-Coach/Teatro/blob/f92db6384ae785adeaa3b301b62ba31f35a5e3d6/Packages/TeatroAudio/README.md>